1. Grafica en el plano cartesiano los vértices de un triángulo A(−2, −1), B(3, −1) y C(1, 4), y únelos con plt.plot para cerrar la figura.

E5_propuesto1.ipynb

IA_Guia05_Matplotlib_Seaborn > Ejercicios_propuestos5 > E5_propuesto1.ipynb > # Carga e importación de Jupyter Notebook

Generate + Code + Markdown | Run All Restart Clear All Outputs ··· Python 3.12.10

```python
# Carga e importación de librerías base
import os
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

# Configuración de estilo global
sns.set_theme(style="whitegrid")

# Coordenadas de los vértices (se repite A al final para cerrar el triángulo)
x_tri = [-2, 3, 1, -2]
y_tri = [-1, -1, 4, -1]

fig, ax = plt.subplots(figsize=(6, 5))

# Graficar líneas y puntos
ax.plot(x_tri, y_tri, marker="o", color="tab:blue", linewidth=2, markersize=8)

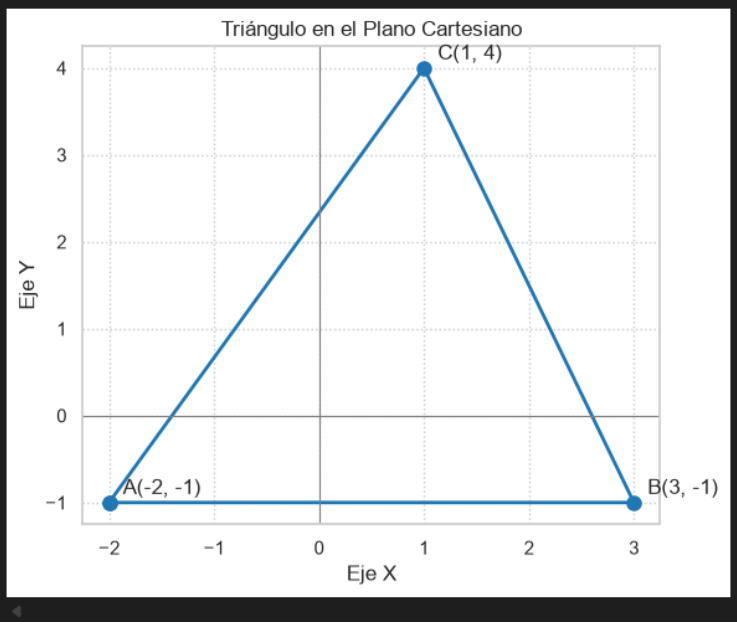
# Etiquetar vértices
etiquetas = {"A": (-2, -1), "B": (3, -1), "C": (1, 4)}
for punto, (x, y) in etiquetas.items():
    ax.annotate(
        f"{punto}({x}, {y})", (x, y), textcoords="offset points", xytext=(8, 5)
    )

# Ejes de referencia en el origen
ax.axhline(0, color="gray", linewidth=0.8)
ax.axvline(0, color="gray", linewidth=0.8)

ax.set_title("Triángulo en el Plano Cartesiano")
ax.set_xlabel("Eje X")
ax.set_ylabel("Eje Y")
ax.grid(True, linestyle=":")

plt.show()
```

[1] ✓ 0.9s Python



2. Grafica en una misma figura $y = x^3$ e $y = -x^2 + 4$ en el intervalo [−3, 3], con leyenda y cuadrícula.

E5_propuesto2.ipynb

IA_Guia05_Matplotlib_Seaborn > Ejercicios_propuestos5 > E5_propuesto2.ipynb > # Carga e importación de Jupyter Notebook

Generate + Code + Markdown | Run All Restart Clear All Outputs ... Python 3.12.10

Generate + Code + Markdown

```python
# Carga e importación de librerías base
import os
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

# Configuración de estilo global
sns.set_theme(style="whitegrid")

# Intervalo [-3, 3]
x = np.linspace(-3, 3, 200)

y1 = x**3
y2 = -x**2 + 4

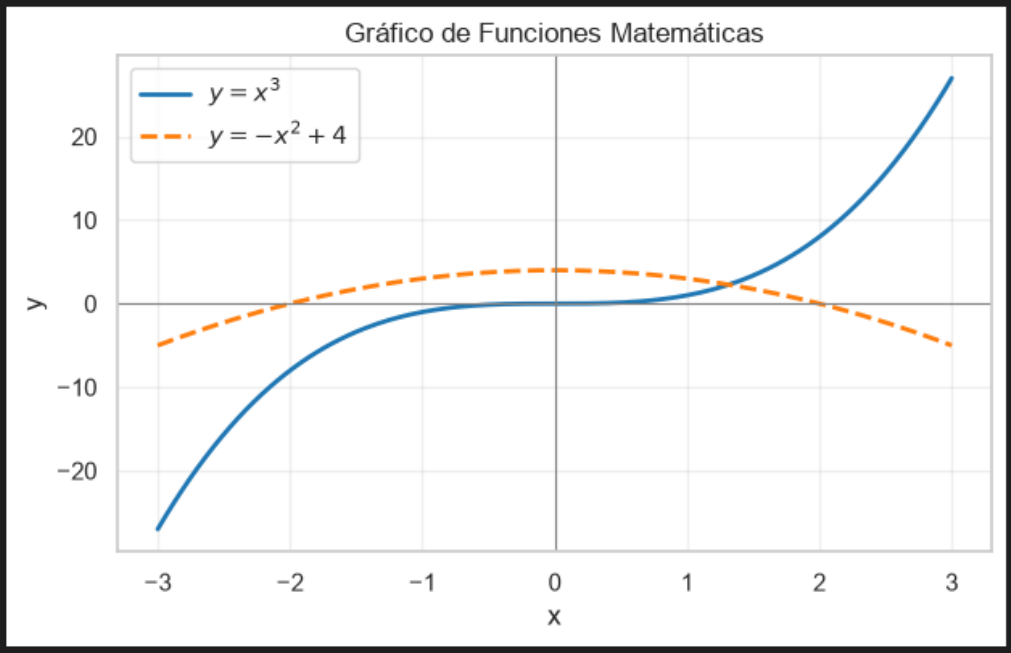
plt.figure(figsize=(7, 4))
plt.plot(x, y1, label=r"$y = x^3$", color="tab:blue", linewidth=2)
plt.plot(
    x,
    y2,
    label=r"$y = -x^2 + 4$",
    color="tab:orange",
    linestyle="--",
    linewidth=2,
)

plt.axhline(0, color="gray", linewidth=0.8)
plt.axvline(0, color="gray", linewidth=0.8)

plt.title("Gráfico de Funciones Matemáticas")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid(True, alpha=0.3)

plt.show()
```

[2] 0.1s Python



3. Crea un histograma con las edades de 100 personas generadas con rng.integers(17, 40, 100). Prueba bins=5 y bins=15 y explica la diferencia.

E5_propuesto3.ipynb

IA_Guia05_Matplotlib_Seaborn > Ejercicios_propuestos5 > E5_propuesto3.ipynb > # Carga e importación de librerías base

Jupyter Notebook

Generate + Code + Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline ... Python 3.12.10

Generate + Code + Markdown

```python
# Carga e importación de librerías base
import os
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

# Configuración de estilo global
sns.set_theme(style="whitegrid")

rng = np.random.default_rng(42)
edades = rng.integers(17, 40, 100)  # Genera 100 edades entre 17 y 39

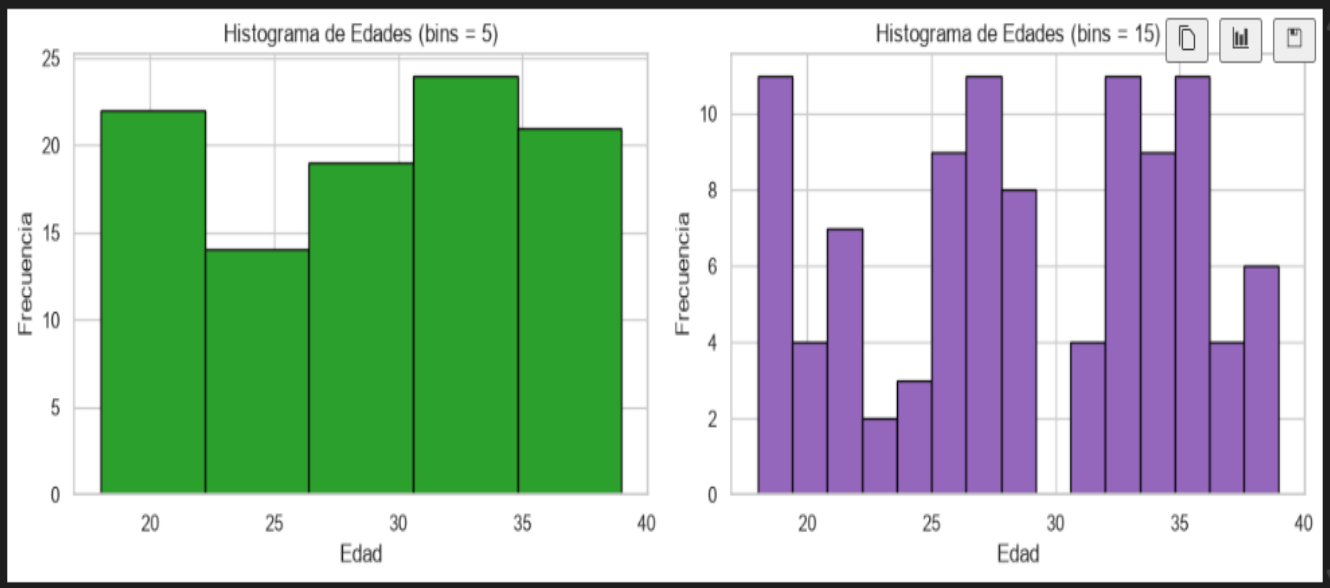
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# Histograma con bins = 5
axes[0].hist(edades, bins=5, color="tab:green", edgecolor="black")
axes[0].set_title("Histograma de Edades (bins = 5)")
axes[0].set_xlabel("Edad")
axes[0].set_ylabel("Frecuencia")

# Histograma con bins = 15
axes[1].hist(edades, bins=15, color="tab:purple", edgecolor="black")
axes[1].set_title("Histograma de Edades (bins = 15)")
axes[1].set_xlabel("Edad")
axes[1].set_ylabel("Frecuencia")

plt.tight_layout()
plt.show()
```

[1] ✓ 1.0s Python



4. Con un DataFrame de ventas por tienda (3 tiendas, 30 días cada una), dibuja un sns.boxplot y responde: ¿qué tienda tiene ventas más estables?

E5_propuesto4.ipynb

IA_Guia05_Matplotlib_Seaborn > Ejercicios_propuestos5 > E5_propuesto4.ipynb > # Carga e importación de librerías base

Jupyter Notebook

Generate + Code + Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline ... Python 3.12.10

```python
# Carga e importación de librerías base
import os
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

# Configuración de estilo global
sns.set_theme(style="whitegrid")

rng = np.random.default_rng(123)

# Generación de datos sintéticos de ventas para 3 tiendas durante 30 días
datos_ventas = {
    "Tienda": ["Tienda A"] * 30 + ["Tienda B"] * 30 + ["Tienda C"] * 30,
    "Ventas": np.concatenate(
        [
            rng.normal(500, 100, 30),  # Tienda A: variabilidad moderada
            rng.normal(500, 25, 30),  # Tienda B: menor dispersión (más estable)
            rng.normal(500, 180, 30),  # Tienda C: alta dispersión
        ]
    ),
}

df_ventas = pd.DataFrame(datos_ventas)

plt.figure(figsize=(7, 5))
sns.boxplot(data=df_ventas, x="Tienda", y="Ventas", palette="Set2")
plt.title("Comparación de Ventas por Tienda (30 Días)")
plt.ylabel("Ventas Diarias ($)")

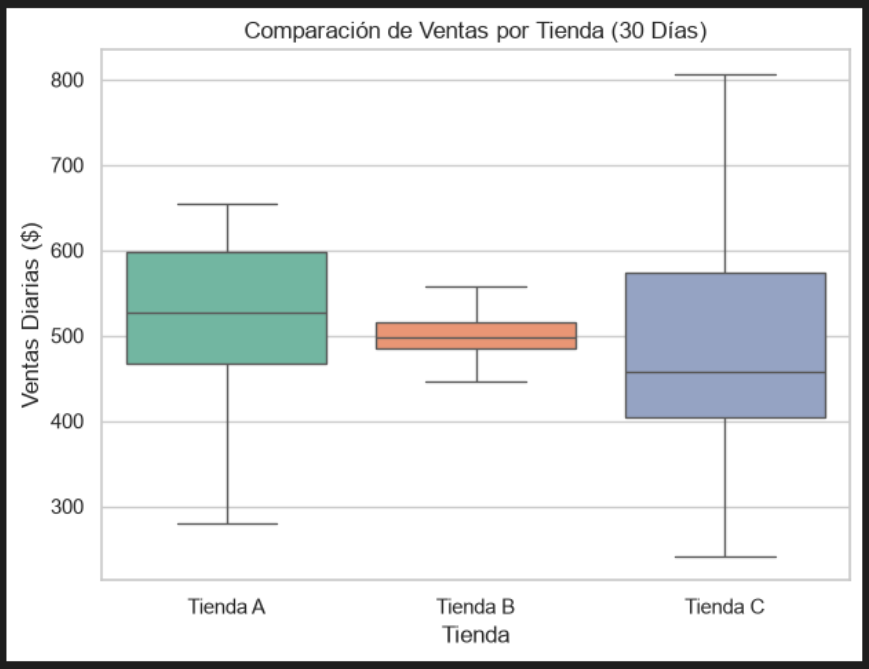
plt.show()
```

[1] ✓ 0.9s Python

```
C:\Users\danie\AppData\Local\Temp\ipykernel_6228\98370308.py:28: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `

  sns.boxplot(data=df_ventas, x="Tienda", y="Ventas", palette="Set2")
```



5. Lee el archivo notas_procesadas.csv de la Guía 01 con Pandas y crea un gráfico de barras con el promedio de cada alumno.

E5_propuesto5.ipynb

IA_Guia05_Matplotlib_Seaborn > Ejercicios_propuestos5 > E5_propuesto5.ipynb > # Carga e importación de librerías base

Jupyter Notebook

Generate + Code + Markdown | Run All | Clear All Outputs | Outline ... Python 3.12.10

Generate + Code + Markdown

```python
# Carga e importación de librerías base
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

# Configuración de estilo global
sns.set_theme(style="whitegrid")

try:
    # Intento de lectura del archivo real
    df_notas = pd.read_csv("notas_procesadas.csv")
except FileNotFoundError:
    # Datos simulados de respaldo si no existe el CSV en el directorio local
    df_notas = pd.DataFrame(
        {
            "Alumno": [
                "Juan",
                "María",
                "Carlos",
                "Ana",
                "Luis",
                "Elena",
                "Pedro",
            ],
            "Promedio": [14.5, 17.2, 11.0, 18.5, 13.0, 16.0, 12.4],
        }
    )

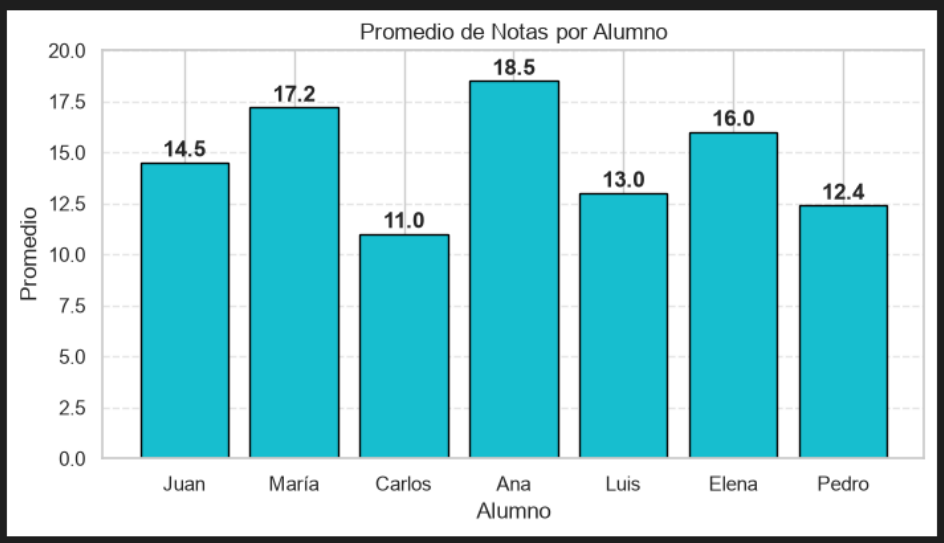
plt.figure(figsize=(8, 4))
plt.bar(
    df_notas["Alumno"],
    df_notas["Promedio"],
    color="tab:cyan",
    edgecolor="black",
)

plt.title("Promedio de Notas por Alumno")
plt.xlabel("Alumno")
plt.ylabel("Promedio")
plt.ylim(0, 20)  # Escala académica
plt.grid(axis="y", linestyle="--", alpha=0.5)

# Agregar el valor numérico sobre cada barra
for i, val in enumerate(df_notas["Promedio"]):
    plt.text(i, val + 0.3, f"{val:.1f}", ha="center", fontweight="bold")

plt.show()
```

[1] 1.0s Python



6. Reto: arma un “mini dashboard” de 2 × 2 con Seaborn sobre un tema de tu elección y expórtalo en PNG (dpi=300) y en PDF dentro de la carpeta graficos.

E5_propuesto6.ipynb

IA_Guia05_Matplotlib_Seaborn > Ejercicios_propuestos5 > E5_propuesto6.ipynb > # Carga e importación de librerías base

Jupyter Notebook

Generate + Code + Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline ... Python 3.12.10

Generate + Code + Markdown

```python
# Carga e importación de librerías base
import os
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

# Configuración de estilo global
sns.set_theme(style="whitegrid")
# Crear directorio de salida si no existe
os.makedirs("graficos", exist_ok=True)

# Dataset para el dashboard: Desempeño y Hábitos de Estudio en Ingeniería
rng = np.random.default_rng(2026)
n = 120

df_dashboard = pd.DataFrame(
    {
        "Horas_Estudio": rng.uniform(2, 20, n).round(1),
        "Asistencia_Pct": rng.uniform(60, 100, n).round(1),
        "Curso": rng.choice(
            ["Python AI", "Bases de Datos", "Redes", "Estructuras"], n
        ),
        "Nota_Final": rng.normal(14, 3, n).clip(0, 20).round(1),
    }
)

# Estructura 2x2
fig, ejes = plt.subplots(2, 2, figsize=(12, 9))

# Subplot 1: Histograma + KDE (Distribución de notas)
sns.histplot(
    data=df_dashboard,
    x="Nota_Final",
    kde=True,
    ax=ejes[0, 0],
    color="tab:blue",
)
ejes[0, 0].set_title("Distribución de Notas Finales")

# Subplot 2: Boxplot (Notas por Curso)
sns.boxplot(
    data=df_dashboard,
    x="Curso",
    y="Nota_Final",
    hue="Curso",
    ax=ejes[0, 1],
    palette="Set3",
)
ejes[0, 1].set_title("Rendimiento Académico por Curso")

# Subplot 3: Scatterplot (Horas de Estudio vs Nota)
sns.scatterplot(
    data=df_dashboard,
    x="Horas_Estudio",
    y="Nota_Final",
    hue="Curso",
    ax=ejes[1, 0],
    s=70,
)
ejes[1, 0].set_title("Relación: Horas de Estudio vs Nota Final")

# Subplot 4: Heatmap de Correlación
corr = df_dashboard[
    ["Horas_Estudio", "Asistencia_Pct", "Nota_Final"]
].corr()
sns.heatmap(corr, annot=True, cmap="Blues", fmt=".2f", ax=ejes[1, 1])
ejes[1, 1].set_title("Matriz de Correlación de Variables")

fig.suptitle(
    "Dashboard Evaluativo de Rendimiento Académico",
    fontsize=16,
    fontweight="bold",
)
fig.tight_layout()

# Guardar imágenes ANTES de plt.show()[cite: 14, 15]
ruta_png = "graficos/mini_dashboard.png"
ruta_pdf = "graficos/mini_dashboard.pdf"

fig.savefig(ruta_png, dpi=300, bbox_inches="tight")  # Exportación PNG 300 DPI[cite: 16]
fig.savefig(ruta_pdf, bbox_inches="tight")  # Exportación Vectorial PDF[cite: 14]

print(f" Archivos exportados con éxito:")
print(f" - PNG (300 DPI): {ruta_png}")
print(f" - PDF Vectorial: {ruta_pdf}")
```

```
    print(f" Archivos exportados con éxito:")
    print(f" - PNG (300 DPI): {ruta_png}")
    print(f" - PDF Vectorial: {ruta_pdf}")

    plt.show()
```

[1] ✓ 2.2s Python

```
Archivos exportados con éxito:
- PNG (300 DPI): graficos/mini_dashboard.png
- PDF Vectorial: graficos/mini_dashboard.pdf
```

